

CDFs and sampling

The running total of probability, and how it turns uniform random numbers into samples.

THE QUESTION

How do we get interval probabilities from the running total $F(x) = P(X \leq x)$, and how does a computer use it to draw samples?

THE INTUITION

The CDF adds up probability from the left, climbing from 0 to 1. Draw a uniform u and find where the climb first reaches height u : likelier values own taller steps, so they come up more often.

KEY TAKEAWAY

$$P(a < X \leq b) = F(b) - F(a).$$

To sample, draw $u \sim$

$\text{Uniform}(0, 1)$ and return $x = F^{-1}(u)$. The same seed always gives the same numbers.

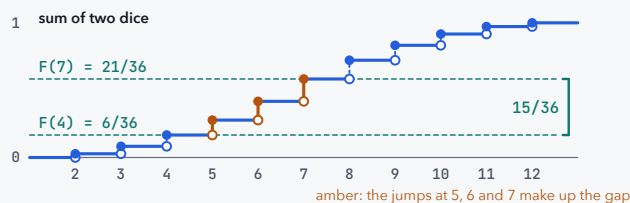
PMF $p(k)$ → cumsum: CDF F → $u \sim \text{Uniform}(0, 1)$ → first x with $F(x) \geq u$ → a draw $x = F^{-1}(u)$

1 $F(x)$ adds up probability from the left



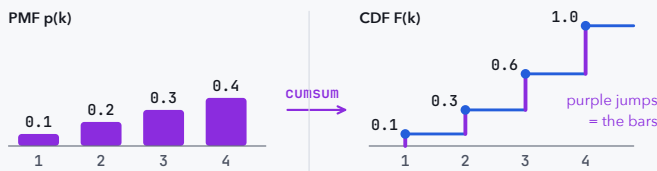
$F_X(x) = P(X \leq x)$ is defined for every real x , not only the values X takes. For a fair die, $F(x) = 0$ for $x < 1$; it jumps by $1/6$ at each face and stays at 1 from 6 on. Between faces it is flat: $F(3) = F(3.7) = 1/2$, as no face lies in $(3, 3.7]$. The filled dot sits on top of each jump, because $F(3)$ already counts $P(X = 3)$.

2 Differences of F give interval chances



$P(a < X \leq b) = F(b) - F(a)$, and $P(X > x) = 1 - F(x)$. For the sum S of two dice, $P(4 < S \leq 7) = 21/36 - 6/36 = 15/36$: the jumps at 5, 6 and 7 (4, 5 and 6 of the 36 pairs). With jumps the ends matter: $P(S \geq 5) = 1 - F(4) = 30/36$, but $P(S > 5) = 1 - F(5) = 26/36$.

3 The jumps are the PMF



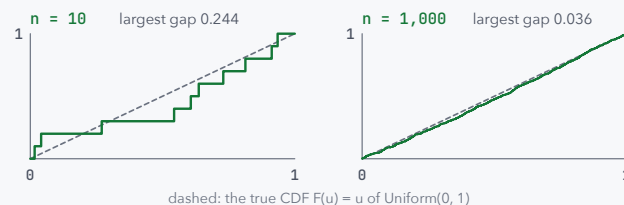
A loaded die shows faces 1 to 4 with chances 0.1, 0.2, 0.3, 0.4. `np.cumsum(p)` gives the running totals 0.1, 0.3, 0.6, 1.0; the way back is $p(k) = F(k) - F(k-1)$, `np.diff(cdf, prepend=0)`. Every CDF never decreases, runs from 0 at $-\infty$ to 1 at $+\infty$, and is right-continuous. A continuous X has no jumps, so $P(X = x) = 0$ and $<$ or $\leq$ give the same answer.

4 The CDF of a maximum is a product



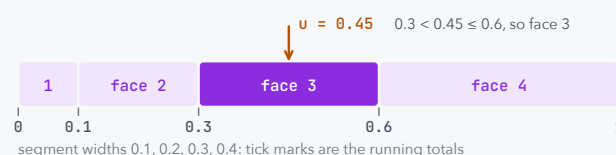
The larger of two dice is at most k exactly when both dice are, so $F(k) = (k/6)^2$: 16 of the 36 pairs for $k = 4$. Differences give the PMF without listing pairs: $P(\max = k) = (k^2 - (k-1)^2)/36 = (2k-1)/36$, from $1/36$ up to $11/36$. Finding $P(Y \leq y)$ first and differencing is the CDF method; it works for any $Y = g(X)$.

5 The empirical CDF tracks the true one



For data $x_1, \dots, x_n$, $\hat{F}_n(x)$ is the share of the x_i that are $\leq x$: sort the data and climb $1/n$ at each point. For uniform numbers on $[0, 1]$ the true CDF is the line $F(u) = u$. With seed 0 the largest vertical gap is 0.244 for 10 samples and 0.036 for 1,000; it shrinks roughly like $1/\sqrt{n}$. In NumPy, plot `np.sort(x)` against `np.arange(1, n + 1) / n`.

6 One uniform number picks a category

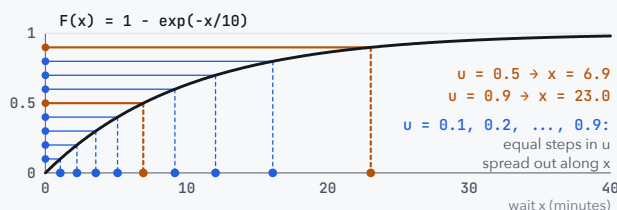


Lay the chances end to end on $[0, 1]$ and draw $u \sim \text{Uniform}(0, 1)$. Face k owns the stretch $(F(k-1), F(k)]$, of length $p(k)$, so it is picked with chance $p(k)$. For $u = 0.45$ the totals are 0.1, 0.3, 0.6, 1.0 and $0.3 < 0.45 \leq 0.6$: face 3. `np.searchsorted(cdf, u)` finds the stretch for a whole array of u at once (code on the next page).

Inverse CDF, quantiles and seeds

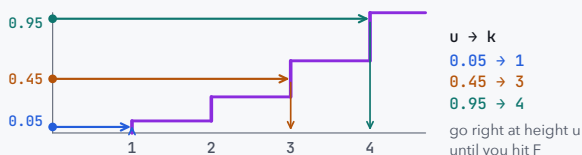
Solve $u = F(x)$ for x to turn uniform numbers into draws. The same inverse gives medians and percentiles.

7 Inverse CDF: solve $u = F(x)$ for x



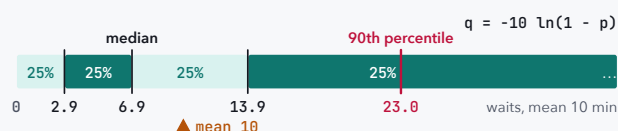
Waits averaging 10 minutes, modelled as exponential with rate $\lambda = 1/10$ per minute, have $F(x) = 1 - e^{-\lambda x}$ for $x \geq 0$ (the density is on the continuous distributions sheet). Solving $u = F(x)$ gives $x = F^{-1}(u) = -\ln(1 - u)/\lambda$. Note $F(10) = 1 - e^{-1} = 0.632$: 63.2% of waits are at most the average.

8 Why it works, for jumps too



For $U \sim \text{Uniform}(0, 1)$, $P(U \leq t) = t$, so $P(F^{-1}(U) \leq x) = P(U \leq F(x)) = F(x)$: the draws have CDF F . With jumps, $F^{-1}(u) = \min\{x : F(x) \geq u\}$, the first value where F reaches height u ; `searchsorted` computes exactly this. $1 - U$ is uniform too, so $-\ln(U)/\lambda$ also works, but `rng.random()` can return 0 (never 1), so $\ln(1 - u)$ is the safe form.

9 Quantiles read F backwards



The p -quantile $q_p = F^{-1}(p)$ has a share p of the probability at or below it: median $q_{0.5}$, quartiles $q_{0.25}$, $q_{0.75}$, 90th percentile $q_{0.9}$. The median wait (6.9) is below the mean (10): a few long waits pull the mean up. A normal has $F(\mu) = 0.5$, so median = mean. A fair die has $F(3) = 1/2$: the rule gives median 3; `np.median` of 1 to 6 averages the middle pair, 3.5.

12 Pause and think

$F(2) = 0.3$ and $F(5) = 0.8$. $P(2 < X \leq 5)$? $0.8 - 0.3 = 0.5$.

Waits average 10 minutes. Share under 10? 63.2%, not half; the median is 6.9.

Can $F(3) = 0.4$ and $F(5) = 0.3$? No: that makes $P(3 < X \leq 5)$ negative.

Same seed, run twice. Same numbers? Yes, exactly: a formula started from the seed.

Loaded die, $u = 0.95$. Which face? 4, since $0.6 < 0.95 \leq 1$.

Seed 1,739,276,086 in the notebook's generator? 0.8099 forever: that x maps to itself.

10 Both samplers in NumPy

```
p = np.array([0.1, 0.2, 0.3, 0.4])
cdf = np.cumsum(p) # 0.1 0.3 0.6 1.0
np.searchsorted(cdf, 0.45) + 1 # 3
rng = np.random.default_rng(0)
u = rng.random(100_000)
face = np.searchsorted(cdf, u) + 1
np.bincount(face)[1:] / 1e5
# 0.102 0.198 0.301 0.400
u2 = rng.random(100_000)
x = -np.log(1 - u2) / 0.1
x.mean(), np.median(x), (x < 10).mean()
# 9.95, 6.88, 0.634
```

Each call moves the generator on, so `u2` is a fresh set of numbers. Shortcuts: `rng.choice(4, p=p)` (faces 0 to 3) and `rng.exponential(10, n)`. NumPy's exponential takes the scale $1/\lambda$ (the mean); PyTorch's `Exponential(rate)` takes λ . PyTorch distributions with an `icdf` method (the normal has one) sample as `dist.icdf(torch.rand(n))`.

11 Random numbers come from a formula

```
x_next = (1664525 x + 1013904223) mod (2^31 - 1)
u = x / (2^31 - 1)
```

seed	42	0.5047	0.1766	0.2970	0.3688	0.9041	
seed 42	0.5047	0.1766	0.2970	0.3688	0.9041	same list	
seed 43	0.5055	0.3580	0.6665	0.5079	0.8872	a new list	

A linear congruential generator updates one integer, $x_{n+1} = (ax_n + c) \bmod m$, and returns $u = x/m$. The course notebook's version uses $a = 1664525$, $c = 1013904223$, $m = 2^{31} - 1$. The seed is the starting x , so the same seed gives the same list and a result can be rerun exactly. This one repeats only after $2^{31} - 2$ steps, yet seeds 42 and 43 start almost together (0.5047, 0.5055). NumPy's `default_rng(42)` scrambles the seed first and uses a stronger generator (PCG64): `.random(3)` gives 0.774, 0.439, 0.859 on every run. Make one generator per experiment and pass it around; the older `np.random.seed(0)` sets one hidden global state that every part of a program shares.

$F(x) = P(X \leq x) \cdot P(a < X \leq b) = F(b) - F(a) \cdot \text{sample } F^{-1}(u), u \text{ uniform} \cdot \text{quantile } F^{-1}(p) \cdot \text{same seed, same numbers}$

Source: the course notebooks "Cumulative distribution function (CDF)" (a categorical with chances (0.1, 0.2, 0.3, 0.4) sampled by `cumsum` and `searchsorted`; inverse-CDF sampling of the exponential), "CDF for discrete random variables" and "Introduction" (a linear congruential generator), nipunbatra.github.io/psdv-teaching. The other examples and all numbers are our own, computed by the sheet's figure script; simulations use seed 0. Next: the continuous distributions sheet.